



Programming in Python (Error Handling)

Dr. Faisal Anwer

Department of Computer Science

Aligarh Muslim University

1

Recap

- Introduction to Module
- Introduction to Package
- Package Creation
- Package Hierarchy

Contents

- Introduction to Error
- Introduction to Exception
- Handling Exceptions
- Problems on Exception

Error

- **Errors** in python can be distinguished in two different kinds: *syntax errors* and *exceptions*.
- **Syntax errors** are the most common kind of error we get while learning Python.

```
>>>for iin range(5): print('Hello world')
```

```
>>>SyntaxError: invalid syntax
```

Exceptions

- A program may be syntactically correct, but it may cause an **error during execution**.
- Errors detected **during execution** are called **exceptions**.
- ***An exception is an event that disrupts the normal flow of the program's instructions.***
- When a Python script encounters a situation that it can't handle, **it raises an exception**.

Exceptions

- An exception is a **Python object** that represents an error.
- When a Python script raises an exception, it **must be handled immediately** otherwise it would **terminate** and come out.

Example

```
A = list(range(10))
o_count=0
e_count=0
for i in range(20):
    if A[i]%2==0:
        e_count+=1
    else:
        o_count+=1
print("Count of even numbers: ",e_count)
print("Count of odd numbers: ",o_count)
```

Without
Exception
Handling

IndexError: list index out of range

Example

```
A = list(range(10))
o_count=0
e_count=0
for i in range(20):
    try:
        if A[i]%2==0:
            e_count+=1
        else:
            o_count+=1
    except:
        print("Index out of range")
        break
print("Count of even numbers: ",e_count)
print("Count of odd numbers: ",o_count)
```

With
Exception
Handling

Handling an exception

- If a code may raise an exception, better to use a **try:** block to defend
- After the try: block, include an **except** statement.
- **Syntax:**

try:

Perform your operations here;

.....

except Exception I:

execute this block.

except Exception II:

execute this block.

except Exception ..

.....

else:

If there is no exception then execute this block.

Handling an exception

- **Example:**

try:

```
fh = open("file1.txt", "r")  
fh.write("my name is XYZ")
```

except IOError:

```
print("Error: can't find file or read data")
```

else:

```
print("Written content in the file successfully")  
fh.close()
```

Handling an exception

- **Except clause with multiple exceptions:**

- You can also use the same *except* statement to handle multiple exceptions as follows:

try:

Perform operations here;

.....

except(Exception1[, Exception2[,...ExceptionN]]):

If there is any exception from the given exception list, then execute this block

.....

else:

If there is no exception then execute this block.

Handling an exception

- **Except clause with no exceptions** can be used to catch all types of exceptions.

try:

You do your operations here;

.....

except:

If there is Exception, then execute this block.

.....

else:

If there is no exception then execute this block.

- **The above type of exception handling is not considered a good programming practice. though,**

Handling an exception

- **The try-finally clause:**

- You can use a **finally:** block along with a **try:** block.

try:

You do your operations here;

.....

Due to any exception, this may be skipped.

finally:

This would always be executed.

.....

Handling an exception

- **The try-finally clause: Example**

try:

```
fh = open("testfile", "w")
```

```
fh.write("This is my test file for exception handling!!")
```

finally:

```
fh.close()
```

Handling an exception

- **Try:** This block will test the excepted error to occur
- **Except:** Here you can handle the error
- **Else:** If there is no exception then this block will be executed
- **Finally:** Finally block always gets executed either exception is generated or not

- **Syntax:**

try:

Some Code...

except:

Optional block

Exception Handling (if required)

else:

Execute if no exception occurs

finally:

Some code(always executed)

Handling an exception

while True:

try:

```
n = int(input("Enter a number: "))
```

```
fh = open("testfile.txt", "w")
```

```
fh.write("This is my test file for exception handling!!")
```

```
sum=0
```

```
for i in range(n+1):
```

```
    sum=sum+i
```

except IOError:

```
    print("File Error")
```

```
    break
```

except ValueError:

```
    print("Invalid entry, Please retry")
```

else:

```
    print("Sum is: ",sum)
```

```
    break
```

finally:

```
    fh.close()
```

Summary

- Discussion on Error
- Discussion on Exception
- Handling Exceptions
- Problems on Exception